

UNIVERSITE DE NICE – SOHPIA ANTIPOLIS

I.U.T. de Nice Côte d'Azur

Département informatique

41, BD Napoléon III – 06200 Nice

RAPPORT DE STAGE POUR L'OBTENTION DU
DIPLOME UNIVERSITAIRE DE TECHNOLOGIE
SESSION 2017-2018

Développement d'une application de gestion de tickets de dépannage et assistance en dépannage informatique.

Présenté par Lucas Barbero



AlloInformatique06
37 bis avenue Henri Matisse 06200, Nice
Sous la direction de :
M. Ludovic Laborde, tuteur entreprise,
Mme. Lise Brenac, tuteur IUT





Remerciement

Je tiens à remercier tout particulièrement mon tuteur en entreprise Monsieur Ludovic Laborde pour m'avoir offert l'opportunité de réaliser un stage dans son entreprise ainsi que pour son implication vis-à-vis de mon apprentissage et de la transmission de son savoir à la fois théoriques et pratiques.

Je remercie également Madame Thuriane Morvan, chargée de communication, pour toute sa considération à mon égard et pour l'ensemble des connaissances qu'elle m'a apportée.

Ce premier stage en entreprise aura été une aventure passionnante ainsi qu'une bonne initiation au monde professionnel.

Enfin, je remercie Madame Lise Brenac d'avoir endossé le rôle d'enseignante encadrante et pour son intérêt quant au bon déroulement de mon stage.

Table des matières

Remerciement.....	3
Résumé.....	6
Abstract	7
Présentation de l'entreprise	8
Allo Informatique 06.....	8
Son Réseau Azur Network	8
Cahier des charges.....	9
Contexte	9
Contexte mission développement 1	9
Contexte mission développement 2	9
Besoins	9
Besoins mission développement 1	9
Fonctionnalités à implémenter.....	10
Besoins mission développement 2	11
Fonctionnalités à implémenter.....	11
Architecture Proposée	11
Réalisation.....	13
Environnement	13
Choix des langages, librairies et IDE.....	13
Java	13
JavaFx	15
JFoeniX.....	17
FontAwesomeFX	17
Gluon Scene Builder	18
MySQL	19
Spring Framework	19
Méthode de développement	21
Modèle de données.....	22
Analyse et conception	22
Création.....	23
API REST.....	24
Définition	25
Fonctionnement.....	25

Création.....	27
Sécurisation de l'API	29
Gestion du SSL et HTTPS	29
Authentification	30
Gestion des rôles	31
Interface cliente	32
Connexion.....	32
Menu Principal.....	33
Création d'une demande	34
Dépannage informatique	36
Notions abordées	36
Solutions informatiques	36
Relation client	37
Business group	38
Difficultés rencontrées	39
Conclusion.....	40
Tables des illustrations.....	41

Résumé

Ce document a pour but de rapporter les missions que j'ai réalisé pendant mon stage de fin d'études de mon DUT informatique. Mon stage se déroulait au sein de l'entreprise Allo Informatique 06 et cela pendant une durée de dix semaines.

AlloInformatique étant une petite entreprise, j'ai évolué au cœur même de l'entreprise et ai ainsi pu suivre le quotidien d'un chef d'entreprise.

L'objectif de ce stage s'inscrit dans la volonté de mon tuteur de ne pas uniquement vouloir m'attribuer une mission de développement « classique » mais également dans la découverte du rôle de chef d'entreprise ainsi que du large panel d'activités de l'entreprise, dont le dépannage informatique occupe une grande place. Le stage se divise donc en deux parties distinctes.

La première partie porte sur la réalisation d'un logiciel de gestion de tickets de dépannage. Cette mission de développement, entièrement réalisée seule et en parfaite autonomie, vise à moderniser et informatiser le mode de traitement des demandes de dépannage client. Son but premier est de réaliser une interface permettant la création des demandes de dépannage côté client ainsi qu'une interface administrateur permettant la gestion de ces requêtes.

Une mission de développement facultative a également été proposée au cours du stage, l'objectif de la seconde mission est de fournir un logiciel portable permettant d'afficher les informations utiles des composants d'un ordinateur lors d'un dépannage informatique.

La deuxième partie du stage a pour but de m'introduire plus profondément au monde de l'entreprise ainsi que dans les activités de dépannage de AlloInformatique en tant qu'assistant. Ce côté du stage beaucoup plus immersif aborde beaucoup de notions techniques relatives à la réparation des ordinateurs, la protection des données des entreprises ainsi qu'à la relation client.

Abstract

The purpose of this document is to report the mission that I realized during my internship at the end of my DUT informatics diploma. My work experience took place within the company AlloInformatique06 and that for a period of 10 weeks. AlloInformatique being a small company, I progressed in the very heart of this one and was able to follow the daily of a company manager.

The aim of this work experience is in the will of my tutor to not only assign a classic development mission to me, but also in the discovering the job of a company manager as well as a wide range of activities of the company, which computer repair took up an important part. The work experience is divided into two distinct parts.

The first part deals with the realization of a ticket management software. This development mission, entirely realized out alone and in complete autonomy, aims to modernize and computerize the method of processing applications of customer troubleshooting. Its primary purpose is to provide an interface for creating client-side troubleshooting requests as well as an administrator interface for handling these requests. An optional development mission was also proposed during the internship, the objective of the second mission is to provide portable software to display the useful information of the components of a computer during a computer repair.

The second part of the internship aims to introduce me more deeply to the world of business and in the troubleshooting activities of AlloInformatique as an assistant. This much more immersive side of the internship addresses a lot of technical notions about computer repair, corporate data protection and customer relationship.

Présentation de l'entreprise

Allo Informatique 06

Allo Informatique 06 créée en 2013 par Ludovic Laborde est une société de services informatiques pour particuliers et professionnels basés à Nice.

En proposant de nombreuses activités comme le dépannage informatique, la maintenance de parc informatique, le conseil ou encore la protection des données, l'entreprise touche de nombreux domaines de l'informatique.

Plusieurs de ses objectifs portent sur la sensibilisation des entreprises, concernant les nouvelles lois RGPD réglementant la protection des données chez les professionnels ayant débuté en mai 2018.

Allo informatique propose un ensemble de solutions permettant de rendre aux normes les entreprises des Alpes Maritimes qui sont encore très peu équipées en termes de protection des données.

Son Réseau Azur Network



Avec un chiffre d'affaire de 1 677 349€ et présidé par Ludovic Laborde, Azur Network est un réseau de dirigeants et professionnels de TPE/PME regroupant un large nombre de secteurs d'activités sur la côte d'Azur. L'objectif de ce business group est de favoriser les échanges entre les entrepreneurs issues de domaines professionnels très différents et ainsi permettre la signature de nouveaux partenariats. Actuellement constitué de 19 membres permanents, le réseau organise des réunions toutes les deux semaines au Negresco à Nice et reste ouvert à toute nouvelle candidature au sein d'Azur Network.

Alloinformatique06 étant un acteur majeur de ce réseau, l'entreprise collabore ainsi avec des secteurs d'activités tels que la communication, le service aux entreprises, la finance, la création de sites web, les ressources humaines...

Cahier des charges

Contexte

Contexte mission développement 1

C'est dans un contexte de développement et de facilitation du mode de gestion des opérations de l'entreprise qu'une majeure partie de la mission s'intègre.

Pour les dépannages d'entreprises professionnelles, Allo Informatique repose sur un système de tickets de dépannage : l'entreprise cliente achète un carnet de tickets dématérialisés à l'entreprise de dépannage et les consomme lorsqu'elle nécessite une intervention de Allo Informatique.

Après chaque intervention le client est déduit d'un certain nombre de tickets correspondant à la durée de l'intervention, et est notifié du nombre de tickets lui restant après dépannage. Si son nombre de tickets est insuffisant un nouveau carnet lui est vendu.

De plus pour organiser un dépannage les clients doivent obligatoirement contacter l'entreprise par mail ou téléphone pour organiser un dépannage.

Contexte mission développement 2

À chaque intervention sur une machine, il est nécessaire pour évaluer l'état de l'ordinateur, de déployer un ensemble de logiciels portables transportés par le dépanneur. Ces logiciels sont souvent trop précis et ne synthétisent pas forcément toutes les informations utiles au dépanneur.

Besoins

Besoins mission développement 1

La gestion des tickets se fait actuellement par mail que ce soit pour l'ajout ou la déduction de tickets, après chaque intervention le client est notifié par mail du nombre de tickets restant et si l'intervention nécessite de racheter un nouveau carnet de tickets.

L'ensemble de ces opérations sont réalisées par l'intervenant manuellement et ne sont mémorisées que par mail. De plus l'intervenant reçoit continuellement des appels ou des mails pour organiser des interventions en entreprise.

L'objectif de la première mission est donc de fournir un système informatisé permettant de gérer la demande d'intervention côté client ainsi que le système de tickets.

Les besoins relatifs à cette mission de développement peuvent se diviser en deux parties coexistantes :

- La partie « entreprise » représentant le demandeur d'intervention, c'est la partie cliente, elle pourra créer de nouvelles demandes de dépannage,
- La partie « administrateur » représentant le dépanneur, il sera l'unique utilisateur de cette partie et pourra gérer et administrer les différentes demandes de dépannages, les entreprises ainsi que leur nombre de tickets.

Fonctionnalités à implémenter

Partie « entreprise » :

- Authentification à l'application avec des logins¹ attribués par l'administrateur,
- Consulter son nombre de tickets restant,
- Créer une demande d'intervention avec un formulaire,
- Consulter l'état de sa demande et la modifier.

Partie « administrateur » :

- Authentification à l'application avec des logins administrateur,
- Répertoire toutes les interventions effectuées avec leurs informations relatives,
- Consulter, créer, modifier, supprimer des interventions,
- Consulter, créer, modifier, supprimer des entreprises clientes,
- Envoyer des récapitulatifs par mail aux clients sur leurs interventions, leur nombre de tickets restant,
- Prélever et recharger un client en tickets.

¹ Identifiants

Besoins mission développement 2

La vérification de l'état de l'ordinateur se fait principalement via le logiciel crystal disk info qui donne de nombreuses informations sur l'état des disques sur lequel le logiciel est déployé. Cependant comme précisé dans le contexte, certaines informations sont inutiles ou ne sont pas présentes dans ce logiciel.

L'objectif de la seconde mission est de fournir un logiciel portable permettant d'afficher les informations essentielles de logiciels comme crystal disk info tout en affichant de nouvelles données qui intéressent l'intervenant.

Fonctionnalités à implémenter

- Analyse et affichages des données S.M.A.R.T² des disques d'un ordinateur pour synthétiser son état
- Analyse et recherche d'informations complémentaires sur un ordinateur

Architecture Proposée

Pour répondre aux besoins de la mission proposée, j'ai opté pour une architecture client-serveur classique.

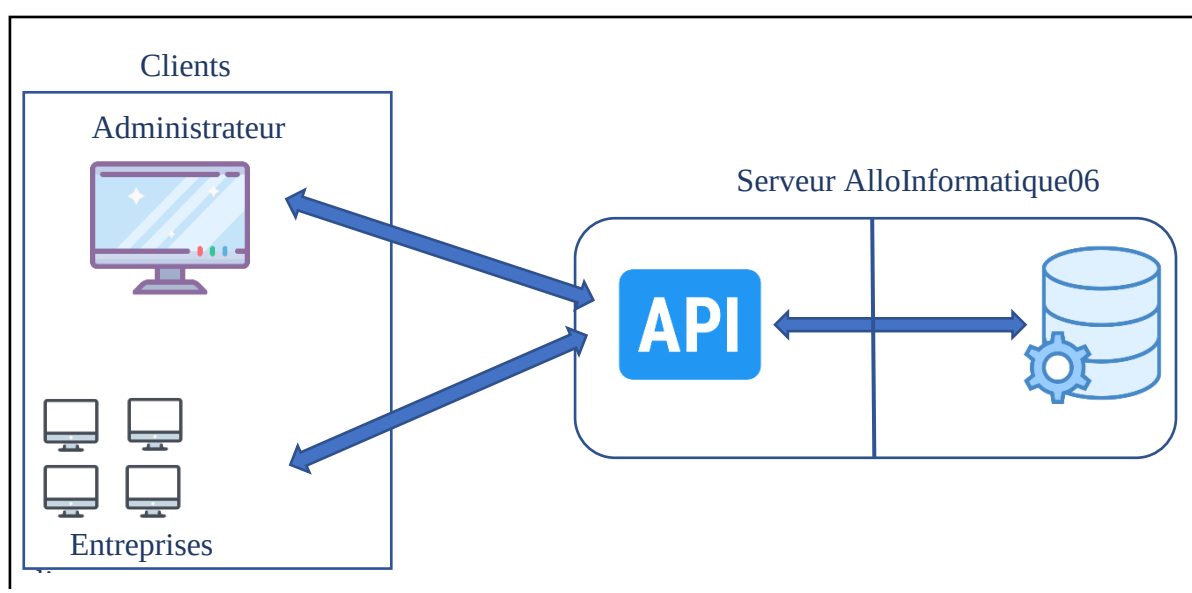


Figure 1 Architecture globale de la mission

² Self-Monitoring, Analysis, and Reporting Technology

Comme son nom l'indique on observe deux parties distinctes :

- Les clients au sens informatique : peuvent être de simples entreprises clientes souhaitant se connecter et réaliser des actions relatives à des demandes de dépannage ou bien l'administrateur voulant consulter ces demandes et ayant un accès total aux données stockées sur le serveur.
- Le serveur, qui hébergera la base de données relationnelle et qui aura pour but de traiter les requêtes émises par les clients.

Afin de répondre aux besoins des parties « entreprise » et « administrateur » explicitées plus haut, j'ai décidé de réaliser deux applications lourdes distinctes, possédant leurs propres interfaces et fonctionnalités.

Cependant ces applications possèdent beaucoup de similarités concernant leurs besoins de communications avec le serveur. J'ai ainsi choisi de réaliser une API, définie dans la suite de ce rapport, me permettant de rajouter une couche logique à mon architecture et de factoriser les communications de mon projet.

L'ensemble des clients émettront des requêtes à cette API qui agit comme une interface faisant le lien entre la partie relative à la présentation des données et celle du stockage et du traitement des données.

Réalisation

Cette partie du rapport explicite l'ensemble des choix réalisés lors de ma mission de développement ainsi que ses résultats. Nous introduisons cette partie par la description de mon environnement de travail.

Environnement

L'ensemble de ma mission de développement a été développée sur ma machine personnelle. Ma mission étant en parfaite autonomie, il m'a été confié de réaliser tous les choix relatifs à l'environnement de développement. Afin de simplifier les tests tout au long du développement, je disposais d'un serveur hébergeant la base de données ainsi que l'API sur ma machine locale.

Choix des langages, librairies et IDE

La totalité des choix pour lesquels j'ai opté possèdent tous une justification que j'ai jugé pertinente compte tenu du temps qui m'était attribué pour le développement de l'application pendant mon stage c'est-à-dire environ 2 tiers de sa période (le dernier tiers étant réservé à la deuxième partie de mon stage).

Java

Les deux applications clientes (entreprise et administrateur) ont été réalisées en Java, étant à l'aise avec ce langage et l'ayant beaucoup étudié au cours de mon DUT, ce choix m'a paru logique pour éviter une phase d'apprentissage d'un langage que je ne connaissais pas ou très peu. L'ensemble de mon code java a été créé sur l'IDE³ IntelliJ IDEA développée par JetBrains, compatible avec l'ensemble des technologies que je vais aborder.



³ Environnement de développement informatique

Java est un langage orienté objet créé en 1995 et développé à ce jour par Oracle, ce langage est très populaire notamment pour ses capacités de portabilité entre les différents systèmes d'exploitation comme Windows, MacOS ou Unix. Cela s'explique par le principe de la machine virtuelle Java ou JVM.

Le code source est premièrement compilé par le compilateur Java, ce qui produit du code machine (Byte Code) qui est ensuite interprété différemment selon le système d'exploitation par la machine virtuelle. Un code source est donc modulable sur plusieurs systèmes.

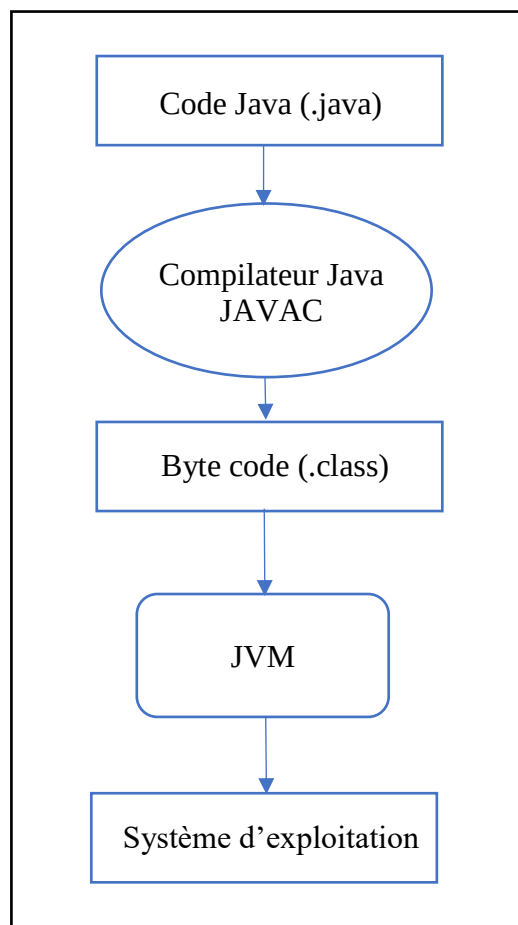


Figure 2 Schéma explicatif de la JVM

JavaFx

JavaFx est la bibliothèque de création d'interface graphique officielle de Java depuis l'apparition de Java 8. Remplaçante de Swing, la bibliothèque plus complète que son prédécesseur est désormais directement disponible dans le kit de développement Java (JDK). Très flexible, JavaFx permet la réalisation d'application lourde, web et mobile.

La représentation simple d'une application JavaFx est composée de trois niveaux :

- Le stage : conteneur principal, assimilé à une une fenêtre, avec des boutons pour la fermer, la minimiser et l'agrandir,
- Une scène : contenue dans un stage, on peut l'associer à une vue ou à un écran de l'application. Cette dernière est amenée à changer au cours de l'utilisation de l'application,

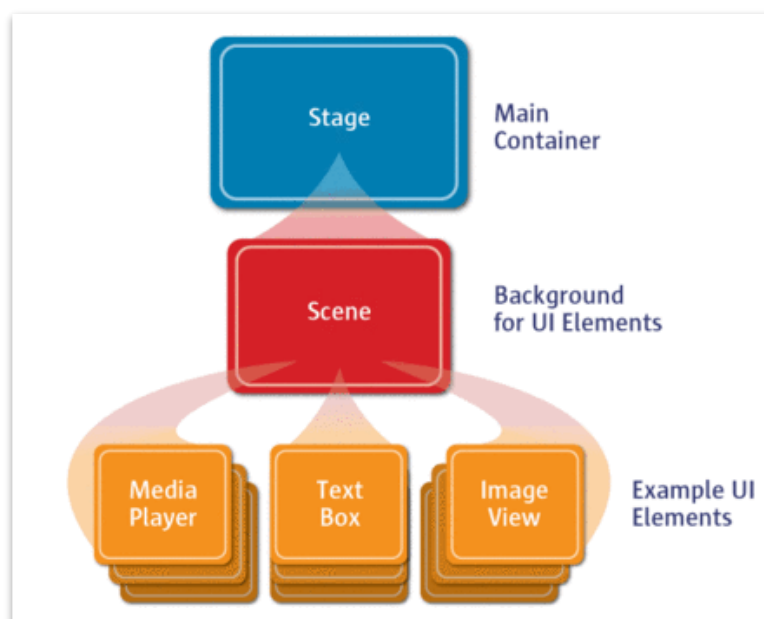


Figure 3 Architecture d'une application JavaFx

- Un ou plusieurs nœuds : ce sont les éléments que l'on manipule le plus dans la création d'interface, on peut les associer à des conteneurs ou bien des éléments plus concrets comme des champs de texte, des images, des listes ...

Afin de décrire et rendre manipulable cette arborescence, JavaFx utilise le format FXML, un document XML⁴ contenant le descriptif de l'arborescence graphique et de l'ensemble des nœuds d'une application. On associe généralement un fichier FXML à une scène de l'application.

```
<?xml version="1.0" encoding="UTF-8"?>

<?import com.jfoenix.controls.JFXButton?>
<?import de.jensd.fx.glyphs.materialdesignicons.MaterialDesignIconView?>
<?import de.jensd.fx.glyphs.octicons.OctIconView?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.AnchorPane?>
<?import javafx.scene.text.Font?>

<AnchorPane prefHeight="600.0" prefWidth="1000.0" stylesheets="@../dark-theme.css" xmlns="http://javafx.com/javafx/8.0.171" xmlns:fx="http://javafx.com/fxml" >
  <children>
    <Label fx:id="ticketLabel" layoutX="469.0" layoutY="213.0" text="Tickets" AnchorPane.bottomAnchor="361.0" AnchorPane.rightAnchor="361.0" />
    <Label fx:id="nomEntrepriseLabel" layoutX="558.0" layoutY="14.0" text="Label" AnchorPane.rightAnchor="14.0" AnchorPane.topAnchor="14.0" />
    <Label layoutX="454.0" layoutY="187.0" text="Il vous reste" textFill="#682525" AnchorPane.bottomAnchor="387.0" AnchorPane.leftAnchor="454.0" />
    <JFXButton fx:id="helpButton" focusTraversable="false" layoutX="238.0" layoutY="310.0" onAction="#handleNewDemande" prefHeight="40.0" />
      <font>
        <Font size="15.0" />
      </font>
    </JFXButton>
    <JFXButton focusTraversable="false" layoutX="564.0" layoutY="310.0" onAction="#handleDemandeOverview" prefHeight="40.0" />
      <font>
        <Font size="15.0" />
      </font>
    </JFXButton>
    <Label fx:id="ticketLabel1" layoutX="469.0" layoutY="239.0" text="Ticket(s)" />
    <MaterialDesignIconView glyphName="HELP" layoutX="296.0" layoutY="388.0" size="45" wrappingWidth="52.99999937415123" />
    <OctIconView glyphName="BOOK" layoutX="626.0" layoutY="363.0" size="45" text="V" />
  </children>
</AnchorPane>
```

Figure 4 Un exemple de fichier FXML

⁴ Extensible Markup Language

Pour finir JavaFx permet également l'intégration du CSS dans ses applications pour accentuer la personnalisation graphique. L'utilisation de toutes ces technologies permet la réalisation d'interfaces modernes et dynamiques.

JFoeniX



JFoeniX est une librairie graphique pour JavaFX rajoutant une multitude de composants material design ainsi que des classes utiles pour la création d'interface.

FontAwesomeFX

FontAwesomeFX est une librairie proposant un large choix d'icônes pour JavaFX sans téléchargements ni traitement de fichiers et répertoires.

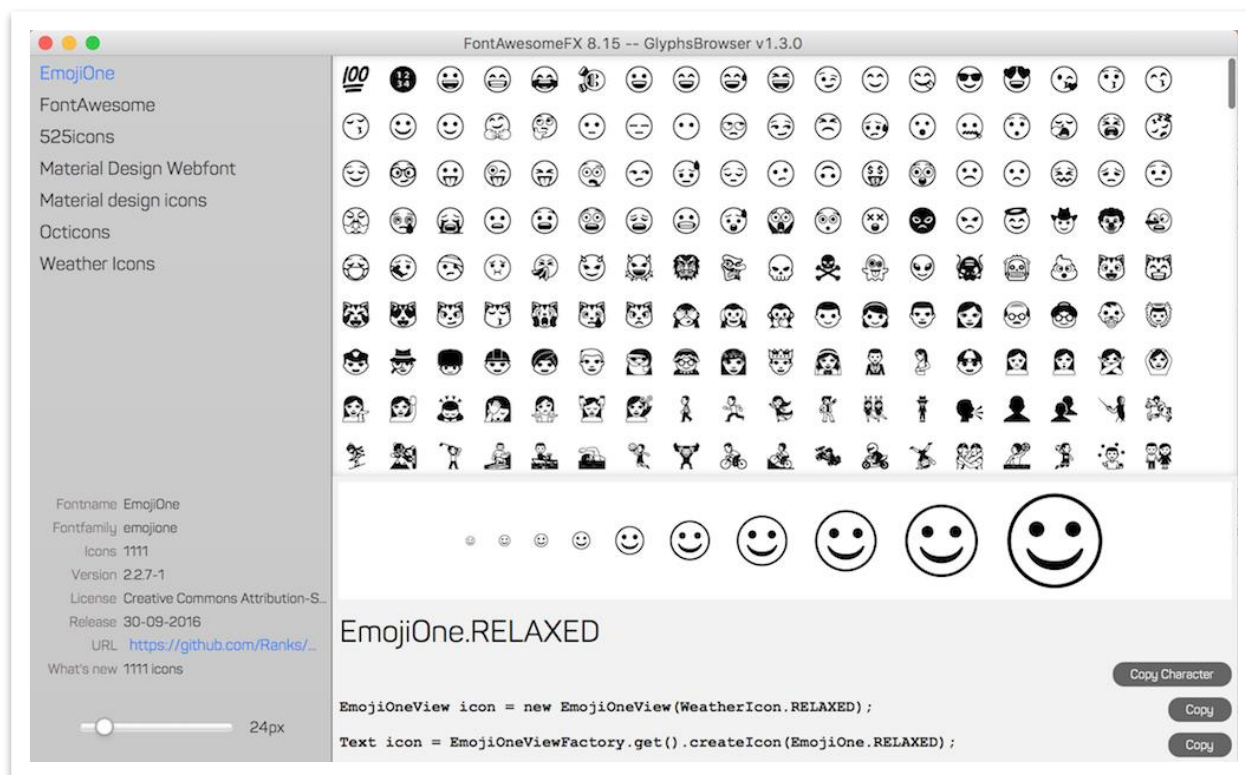


Figure 5 Interface de recherche des icones FontAwesomeFX

Gluon Scene Builder

Gluon Scene Builder est un logiciel de création d'interface JavaFX en drag en drag. Il permet de faciliter grandement la gestion de l'arborescence graphique JavaFX et génère automatiquement les FXML. Il est compatible avec les librairies mentionnées plus haut comme JFoenix. J'ai réalisé l'ensemble des scènes de mon projet avec cette interface.

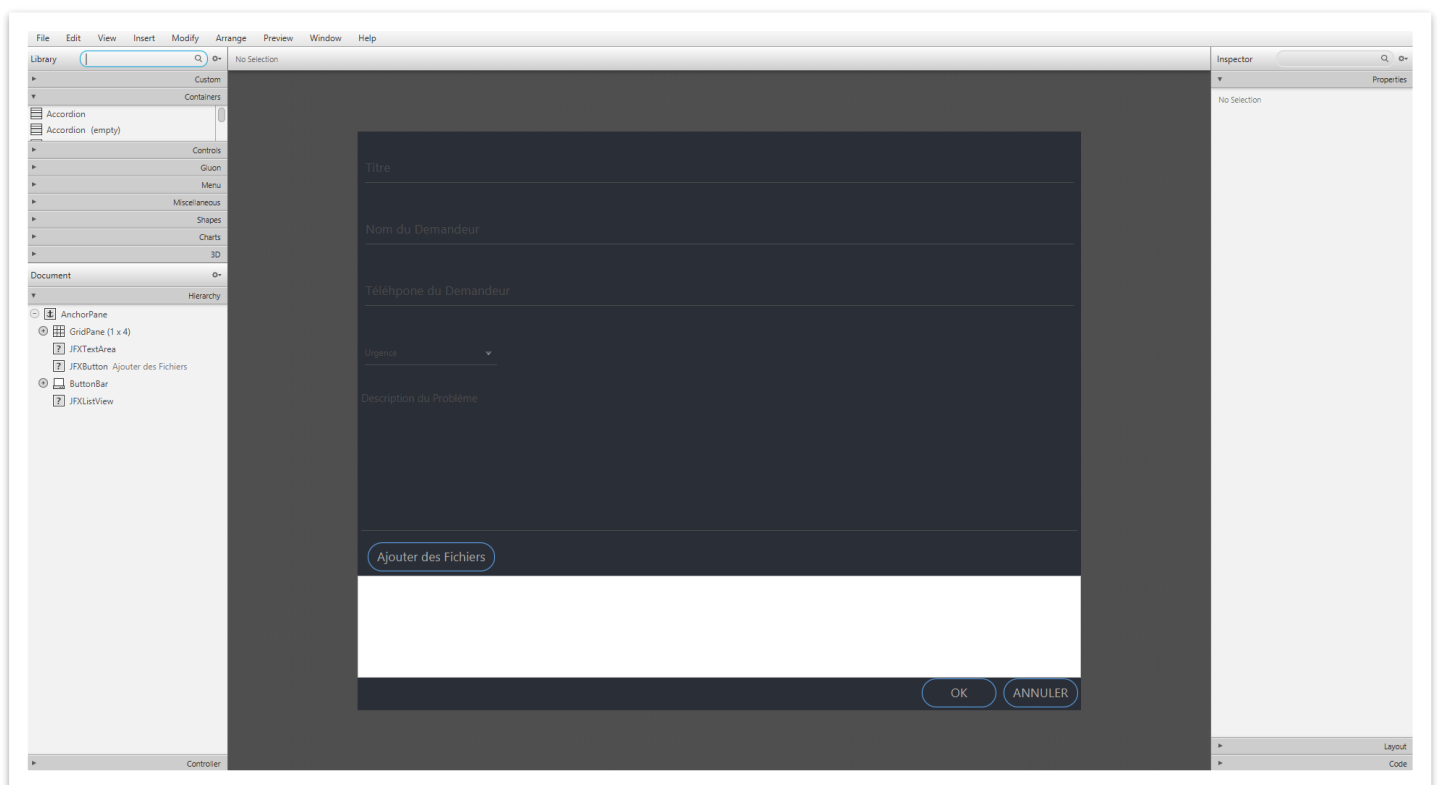


Figure 6 Le scene builder permettant la réalisation d'interface JavaFX

MySQL



L'ensemble des informations stockées seront contenues dans une base de données MySQL qui sera premièrement installée en local puis exportée sur le serveur final. J'ai choisi ce SGBD ⁵car il m'était familier, le choix du SGBD important peu car le modèle de base de données du projet est simpliste et peu de données seront traitées.

MySQL est un serveur de base de données relationnelles SQL c'est-à-dire que l'ensemble des données sont organisées dans des tables à deux dimensions, chaque table possédant une ou plusieurs relations avec d'autres tables d'une même base. MySQL est un des systèmes de gestion de base de données les plus utilisés au monde, reconnu pour son efficacité il est en concurrence avec d'autre SGBD comme Oracle.

Spring Framework



Spring est un framework ⁶java très populaire permettant le développement de l'infrastructure d'applications dans le langage d'Oracle. Spring s'impose comme une alternative de Java Enterprise Edition ou Java EE qui permet également le développement d'applications déployées sur un serveur.

⁵ Système de gestion de base de données

⁶ Ensemble de composants logiciels

Spring possède de nombreuses fonctionnalités regroupées en différents modules. Mon utilisation de Spring se limite au développement d'une RESTful API définie plus tard dans la suite de ce rapport.

Ce framework est une solution comme une autre pour réaliser une API, mon choix s'est porté sur lui pour sa forte documentation, la communauté autour du projet ainsi que pour sa popularité. De plus l'utilisation de Spring me permettait de développer la totalité de mon projet en Java pour le côté client et serveur ce qui n'aurait pas été le cas si j'avais opté pour une autre solution comme le PHP par exemple.

Spring nécessite un certain temps d'apprentissage des différents concepts et modules mais se révèle être très pratique et évolutif une fois maîtrisé.

Les modules que j'ai utilisé pour la réalisation de mon API sont les suivants :

- Spring Boot : Spring Boot est le module principal pour réaliser une application web Spring préconfigurée. Il configure automatiquement les dépendances liées aux autres bibliothèques et modules de Spring et permet ainsi de réduire le temps de configuration pour développer plus vite l'application principale et ainsi augmenter la productivité. Spring Boot intègre également des serveurs http tel que Tomcat pour faciliter le test des applications web produites.
- Spring data JPA : Spring data JPA est une spécification du module Spring data, il permet une implémentation plus simple de l'accès aux bases de données pour une application Spring. De plus il intègre l'api Java de persistance des données (JPA). La persistance des données consiste à l'élaboration d'un mapping objet-relationnel c'est-à-dire une correspondance entre un modèle objet Java et un modèle relationnel SQL. Ce principe rajoute ainsi une couche d'abstraction entre ces deux concepts. L'intérêt de ce module est de s'affranchir du code redondant faisant le lien entre les comportements des objets et les requêtes SQL auxquels ils sont associés. On manipulera uniquement des objets dit « métiers » que l'on ajoutera directement à la base de données à partir de méthodes.
- Spring security : Spring security est un module de sécurité portant sur les applications Spring, ce module est indispensable pour les communications web qui nécessitent d'être sécurisées sous peine d'être interceptées et analysées.

Méthode de développement

Le cycle de vie du logiciel c'est-à-dire toutes les étapes de conception et de développement a suivi un cycle de développement en V. La réalisation de l'application ne couvrant pas la totalité du stage, les phases de conceptions ont été diminuées au maximum au profit de phases de tests fonctionnelles réalisées tout au long du développement. Des phases de conceptions minimales ont le désavantage de provoquer plus d'erreurs lors du développement mais permettent la production de résultats plus tôt.

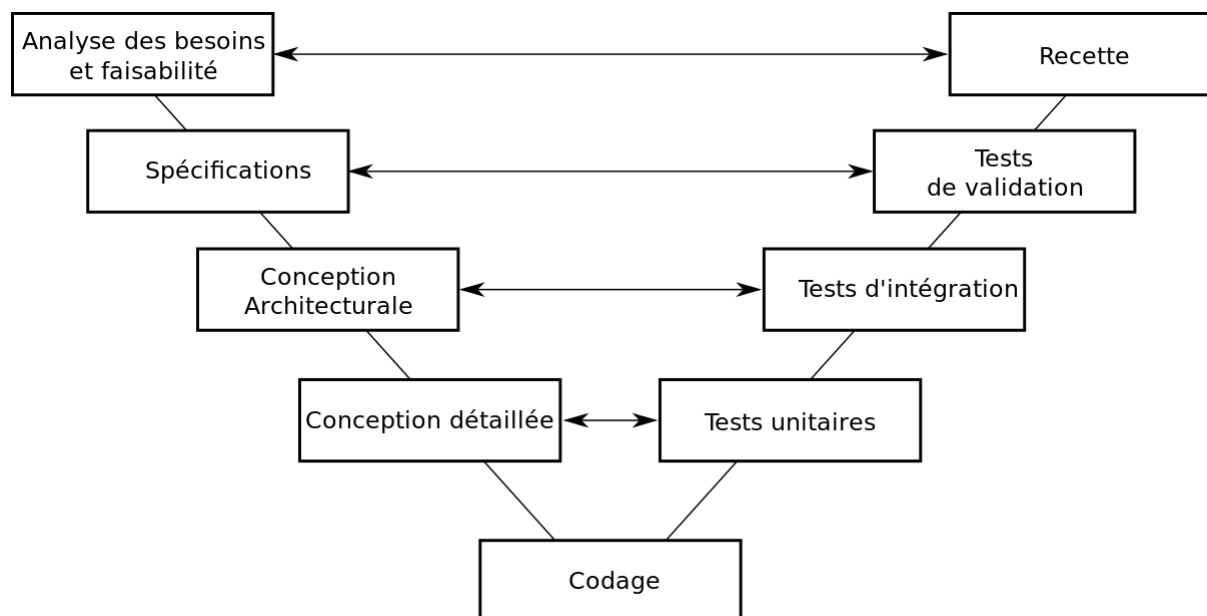


Figure 7 Schéma de réalisation d'un cycle en V

Modèle de données

Analyse et conception

La première étape de la réalisation a été de définir quelles données devaient être stockées et quelles étaient leurs spécificités. Après l'analyse des besoins du cahier des charges j'ai pu relever les observations suivantes :

- Il est nécessaire de stocker les données relatives aux informations des entreprises clientes qui souhaiteront faire des demandes de dépannage,
- Chaque demande de dépannage a besoin d'être archivée avec différentes informations comme la date de demande, le statut, l'entreprise demandeuse ...,
- Pour étoffer sa demande l'entreprise peut fournir des fichiers décrivant son problème.

Ces observations nous ont emmené à la réalisation de trois tables de base de données différentes :

- La table « Entreprise » qui regroupe toutes les informations des entreprises clientes,
- La table « Demande » qui regroupe toutes les informations des demandes des entreprises,
- La table « Fichier » qui regroupe les informations des fichiers fournies avec une demande.

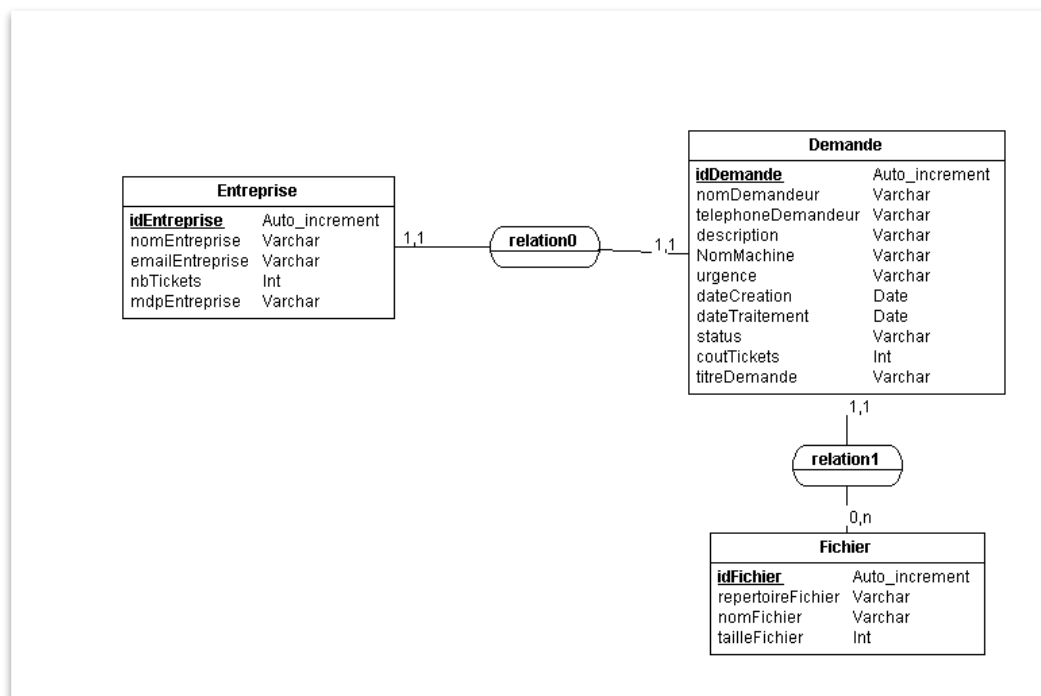


Figure 8 Schéma entité/association initial

Durant le développement de l'application et pour un souci de sécurité, une nouvelle table est venue s'ajouter au modèle de données, il s'agit de la table « Rôle » qui spécifiera si l'entreprise est un utilisateur classique ou administrateur, choix que je développerai plus amplement dans la partie « sécurisation » de ce rapport.

Il est également important de noter que j'ai choisi de stocker dans la table « Fichier » uniquement les informations relatives aux documents (nom, taille, chemin d'accès sur le serveur) et non pas les données des fichiers eux même. Les fichiers issus des demandes seront sauvegardés dans un répertoire dédié du serveur et seront transférés lors de la création de la demande.

Création

Au cours du projet je n'ai pas eu à manipuler le langage SQL, grâce à l'architecture de mon projet Spring il m'a suffi de fournir un modèle objet respectant les normes JPA pour générer automatiquement des requêtes SQL créant les tables de ma base de données. Pour effectuer cela il faut tout d'abord lier notre application Spring côté serveur avec notre base de données dans un fichier de propriétés.

```
spring.jpa.hibernate.ddl-auto=none  
spring.datasource.url=jdbc:mysql://localhost:3306/test?useSSL=true  
spring.datasource.username=admin  
spring.datasource.password=admin
```

Figure 9 Configuration de Spring avec la base de données mysql

Une fois ce lien établi on peut concevoir notre modèle de base de données non pas en créant les tables avec des requêtes SQL mais en utilisant des objets java.

```
@Entity
@JsonIgnoreProperties
public class Entreprise {
    @Id
    @GeneratedValue(strategy= GenerationType.AUTO )
    private long idEntreprise;
    private String nomEntreprise;
    private String emailEntreprise;
    private String password;
    @ManyToOne
    @JoinColumn(name="idRole")
    @JsonIgnoreProperties("entrepriseSet")
    private Role role;
    @OneToMany(fetch = FetchType.EAGER, mappedBy="entreprise", cascade = CascadeType.MERGE)
    @JsonIgnoreProperties("entreprise")
    private Set<Demande> demandesEntreprise;

    private int nbTickets;
}
```

Figure 11 La classe Entreprise annotées des normes JPA

La définition de la classe s'accompagne d'un ensemble d'annotations précédées d'un « @ » qui précisent le rôle des attributs de l'objet dans la génération de la table de base de données. J'utilise une configuration qui utilise par défaut tous les attributs d'un objet comme colonnes d'une table SQL. Il est cependant nécessaire de préciser les informations telles que la clé primaire d'une table⁷ ou le type de relation (cardinalité) avec les autres tables.

Dans l'exemple ci-dessus, le lancement de l'application pour la première fois générera la table « Entreprise », constituée d'autant d'attributs que son objet métier associé. Cette méthode de génération est très flexible et permet la modification d'un schéma de base de données de manière très simple, si la structure de l'objet métier est modifiée, la structure de la base de données le sera également. Ces fonctionnalités m'ont été très utiles lorsque des modifications s'imposaient et permettront d'assurer l'aspect évolutif de l'application.

API REST

Une fois que le modèle de données a été réalisé j'ai débuté le développement de l'élément principal de mon projet qui était la réalisation d'une API REST grâce au framework Spring. Cette partie du rapport vise à définir et expliquer son fonctionnement.

⁷ Ensemble d'attributs qui permet de distinguer un enregistrement d'un autre

Définition

Le terme API signifie « Application Programming Interface » ou interface de programmation applicative, elle permet d'établir des connexions entre plusieurs logiciels pour échanger des données. Sous forme de bibliothèque logicielle ou de service web, elle a un rôle d'intermédiaire entre une application utilisatrice et un ensemble de fonctionnalités issues d'un autre programme. Ainsi une application peut avoir recours à des fonctionnalités grâce à une API sans pour autant avoir accès à leur implémentation et leur code source.

Il existe plusieurs types d'API dont les deux plus connues sont SOAP « Simple Object Access Protocol » que nous n'aborderons pas dans ce rapport et les API dites REST, c'est celle que j'ai décidé d'adopter.

Le terme REST correspond à « Representational State Transfer », une API REST ou RESTful API repose sur le protocole HTTP⁸ et se calque sur le modèle web pour ses échanges.

Fonctionnement

Dans le cadre du projet, les applications clientes (entreprises et administrateur) auront recours aux fonctionnalités liées à la base de données, comme par exemple ajouter, modifier et supprimer des enregistrements d'une table ou envoyer des fichiers vers le serveur.

L'API sera un webservice fonctionnant sur le serveur. C'est-à-dire un programme sur le serveur à l'écoute de requêtes provenant des clients. Il servira d'interface entre la base de données et ces applications.

Le schéma de fonctionnement est le même quel que soit le type d'application cliente :

- L'application émet une requête HTTP à l'API sur un URL⁹ spécifique. Il existe plusieurs types de requête HTTP au nom de « get », « post », « put » et « delete »,
- En fonction de l'URL et du type de requête, l'API exécute une méthode spécifique et communique avec la base de données située sur le même serveur,
- L'API répond avec une requête HTTP décrivant le statut de la demande.

⁸ Hypertext Transfer Protocol, c'est un protocole de communication client-serveur

⁹ Uniform Resource Locator, l'adresse publique de l'API

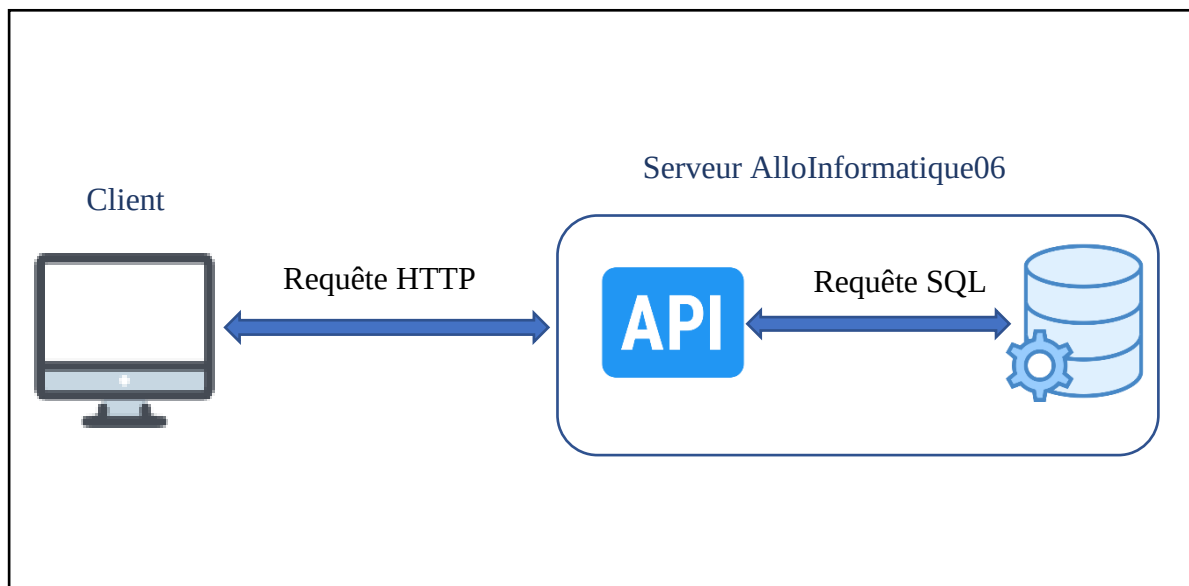


Figure 12 Représentation des échanges dans l'architecture

L'ensemble des requêtes émises peuvent contenir des données correspondantes à des objets ou des fichiers. Les données délivrées par le client au serveur ou inversement seront formatées en JSON (JavaScript Object Notation), format couramment utilisé dans les API REST. Le format est très simple d'écriture et est aisément analysable par les machines.

Quand on souhaite par exemple ajouter une entreprise, cela nécessite d'envoyer toutes ses caractéristiques à l'API pour qu'elle puisse les transmettre à la base de données. Les objets java ne pouvant pas être transmis tels quels via le protocole http, il est obligatoire de les sérialiser en une chaîne de caractères formatées en JSON.

Tout objet métier sera ainsi converti dans son modèle JSON qui sera envoyé à l'API et qui effectuera ensuite le processus inverse. A partir d'un JSON donné elle le reformatera en un objet java de type connu et pourra l'ajouter à la base de données grâce au modèle de persistance de données.

```
{
  "idEntreprise": 3,
  "nomEntreprise": "admin",
  "emailEntreprise": "admin",
  "password": "$2a$10$j71psGwmtntvxqYDkPhXCeZmAP6DVcAD6ajF3x0SbZKKrspqP4MBm",
  "role": {
    "idRole": 1,
    "nomRole": "admin"
  },
  "demandesEntreprise": [
    {
      "idDemande": 8,
      "nomDemande": "11",
      "nomDemandeur": "11",
      "telephoneDemandeur": "1",
      "description": "11",
      "nomMachine": "11",
      "urgence": "MOYENNE",
      "dateCreation": "2018-06-08",
      "dateTraitement": "2018-06-14",
      "status": "NTRAITE",
      "coutTickets": 1
    }
  ],
  "nbTickets": 155
}
```

Figure 13 Format JSON d'une instance de la classe Entreprise

Création

La réalisation de l'API REST en utilisant Spring repose sur deux étapes fondamentales, la première consiste à la création des JpaRepository.

Les JpaRepository sont des interfaces génériques java dont les méthodes sont implémentées automatiquement à la compilation par Spring Data JPA. Ces interfaces sont responsables de la conversion des instances des objets métiers en enregistrements SQL. Les méthodes qu'elles génèrent utilisent des requêtes SQL, ce sont les interfaces qui communiquent donc avec la base de données. Il est également possible de réaliser des requêtes personnalisées vers la base de données avec des prototypes de méthodes possédant des mots clés spécifiques.

```
public interface EntrepriseRepository extends JpaRepository<Entreprise, Long> {  
    Entreprise findByEmailEntreprise(String emailEntreprise);  
}
```

Figure 14 Déclaration du JpaRepository portant sur l'objet Entreprise

La syntaxe `findByEmailEntreprise` permet d'implémenter automatiquement une méthode du repository recherchant un enregistrement dans la table « Entreprise » qui possède la valeur renseignée en argument pour l'attribut « emailEntreprise ». De plus cette méthode retournera le résultat de la requête sous forme d'objet.

La deuxième étape consiste à la définition des Controller qui sont les classes qui régissent l'écoute et le traitement des requêtes HTTP par le serveur. Dans mon cas, chaque classe métiers possède un contrôleur qui lui-même possède en attribut un JpaRepository correspondant à son type. Les contrôleurs Spring permettent de définir un mapping, c'est-à-dire quelles actions effectuer en fonction de l'URL de destination et du type d'une requête HTTP.

Par exemple, si l'adresse de l'api est disponible sur <https://allo-informatique06.fr/api/> il est possible d'associer à l'adresse <https://allo-informatique06.fr/api/entreprises/> l'ensemble des requêtes portant sur la table entreprise. Le mapping que j'ai réalisé pour la REST API est le suivant :

- Les requêtes http GET sur <https://allo-informatique06.fr/api/entreprises> retournerons l'ensemble des entreprises,
- Les requêtes http GET sur <https://allo-informatique06.fr/api/entreprises/{id}> retournerons l'entreprise d'id {id},
- Les requêtes http DELETE sur <https://allo-informatique06.fr/api/entreprises/{id}> supprimerons l'entreprise d'id {id} de la base de données,
- Les requêtes http POST sur <https://allo-informatique06.fr/api/entreprises> créerons une entreprise dans la base de données avec l'entreprise au format JSON transportée dans la requête,

- Les requêtes http POST sur <https://allo-informatique06.fr/api/entreprises/{id}> modifieront l'entreprise d'id {id} dans la base de données avec les caractéristiques de l'entreprise au format JSON transportée dans la requête.

Sécurisation de l'API

Une fois les fonctionnalités principales de l'API réalisées, il était possible de recevoir, de modifier et de supprimer l'ensemble des données stockées à travers l'interface. Cependant l'état actuel de l'application web posait des problèmes de sécurité :

- L'API était publique : si elle était hébergée sur un serveur public en l'état, tout le monde y aurait accès et pourrait accéder à des données privées,
- On ne distinguait pas les requêtes que seul un administrateur pouvait réaliser avec celle qu'une entreprise cliente effectuait,
- L'API ne supportait pas l'utilisation de protocoles sécurisés.

L'observation de ces failles a conduit au développement de solutions permettant de compléter la structure de l'application web et sa sécurisation. Un des outils principaux pour mener à bien cet objectif a été le module « Spring Security » qui fournit notamment la gestion des authentifications et des autorisations pour une application Spring.

Gestion du SSL et HTTPS

Le SSL pour « Secure Socket Layer » est un protocole de sécurité très répandu assurant la sécurité et le cryptage des échanges entre deux machines sur internet ou dans un même réseau. Il agit comme un canal entre les deux machines empêchant toutes interceptions des échanges par un tiers.

L'utilisation d'un protocole SSL sur un serveur permet le passage du protocole HTTP à HTTPS, le « S » signifiant « sécurisé ». HTTPS est une extension de HTTP garantissant une connexion sécurisée à un serveur.

Le SSL fonctionne avec des certificats électroniques délivrés par des autorités tierces auxquelles on peut faire confiance. Ce certificat garanti aux demandeurs de connexion que le serveur est bien celui qu'il prétend être et que la connexion établie sera chiffrée et sécurisée.

Pour sécuriser une application Spring avec du SSL il est nécessaire de lui indiquer dans un fichier de configuration où sont stockées les informations du certificat sur le serveur. De plus il faut forcer l'utilisation du SSL pour la connexion avec la base de données.

Après avoir installés le SSL sur le serveur et avoir configuré Spring, une dernière opération a été réalisée, il s'agit de la redirection des requêtes HTTP vers HTTPS. En effet par défaut les requêtes HTTP pointent vers le port 80 d'un serveur alors que les requêtes HTTPS se dirigent vers le port 443. La dernière étape consiste donc à rediriger les requêtes du port 80 non sécurisées vers le port 443 qui lui est protégé.

Authentification

Malgré que l'API garantit des échanges en toute sécurité, elle n'imposait aucune contrainte sur qui pouvait y accéder. En effet l'utilisation de l'application se résumant à l'administrateur et aux entreprises clientes, il n'était pas logique que quiconque possédant le lien de l'API puisse y faire appel. L'utilisation de l'API devait donc être limitée aux clients et administrateurs.

La solution la plus simple et la plus adaptée pour l'envergure de l'application était une authentification basique. Son principe est très simple, l'ensemble des entreprises sont enregistrés avec un login associé à leur email et un mot de passe, déterminé par l'administrateur et stockée de manière cryptée dans la base de données.

A chaque demande est ajoutée dans l'entête HTTP de la requête un champ « Authorization » suivi des identifiants du demandeur encodés en base 64. Suite à une demande, l'API va consulter l'entête de la requête HTTP, décoder ce champ et vérifier que les identifiants sont corrects et correspondent avec une entreprise de la base de données. Si les identifiants sont corrects la requête est effectuée dans le cas contraire elle est rejetée.

Ce mécanisme est réalisé dans Spring par le biais d'un point d'entrée d'authentification ou « Basic Authentication Entry Point ». Cette classe intercepte toutes les requêtes en direction de l'API et vérifie leur entête. A partir de cette entête le point d'entrée décode l'identifiant de l'entreprise fourni et va appeler un service de vérification.

Ce service qui possède un accès au JpaRepository des entreprises va effectuer une recherche par email dans la base de données pour vérifier que l'entreprise existe. Si tel est le cas, le service va crypter le mot de passe fourni dans l'entête et le comparer avec celui stocké dans la base de données. Si les deux mots de passe cryptés sont identiques, les identifiants renseignés dans l'entête de la requête sont justes.

Gestion des rôles

Désormais seuls les utilisateurs répertoriés dans la base de données peuvent effectuer des demandes à l'API, cependant on ne pose pas de limite quant aux types de requêtes réalisables par un utilisateur « classique » ou administrateur. Pour régler ce dernier problème de sécurité, il a été nécessaire de rajouter une table dans le modèle relationnel de notre base de données. Cette table se nomme « Rôle » chaque entreprise possède un rôle, il en existe deux actuellement :

- Le rôle administrateur : il peut réaliser toutes les requêtes,
- Le rôle user : il peut uniquement réaliser des requêtes portant sur la création et la modification de demandes de dépannage et de fichiers.

Pour intégrer cette nouvelle composante à Spring, il a été seulement nécessaire de modifier le service de vérification utilisateur. Après vérification des identifiants on analyse le rôle de l'utilisateur qui souhaite se connecter, en fonction du rôle on bloque certain mapping de l'API. Si la requête se dirige vers un mapping dont elle ne possède pas les droits d'accès une erreur est retournée.

A noter que le compte administrateur est ici modélisé comme une entreprise classique qui possède le rôle administrateur. Ce compte admin est créé lors de l'initialisation de la base de données. Ce système de rôle à l'avantage d'être très simple à intégrer et peut facilement évoluer dans le temps si l'introduction de nouveaux rôles et droit d'accès sont requis.

Interface cliente

La dernière partie de la réalisation de l'application a été le développement de l'interface graphique avec JavaFx, à ce jour seul l'interface pour les entreprises clientes a été réalisée. Le développement de l'IHM¹⁰ après la réalisation de l'API et de sa sécurisation a permis une intégration directe de tous les composants déjà réalisés.

Connexion

Le premier écran de l'application entreprise est un écran de connexion, comme spécifié plus haut, chaque entreprise possède un email et un mot de passe.

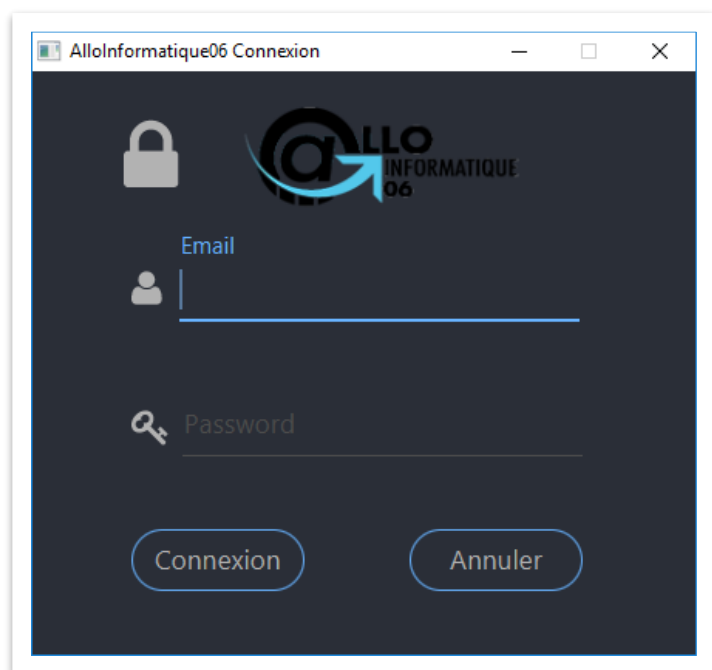


Figure 16 Interface d'identification

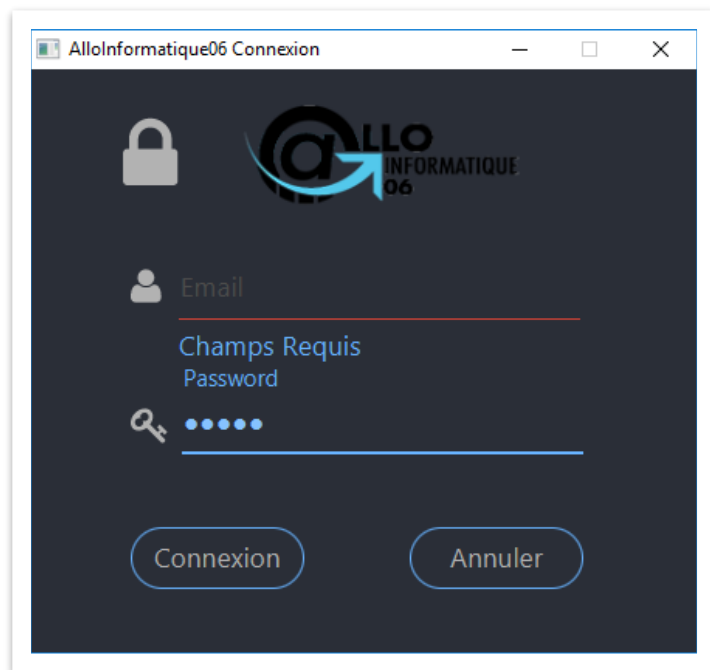


Figure 15 Gestion des champs obligatoires

Le formulaire de connexion vérifie que tous les champs sont complétés. Une fois le bouton de connexion activé, l'application tente d'envoyer une requête HTTP possédant en entête les identifiants fournis sur le formulaire vers un mapping « logging » testant uniquement les champs renseignés. Si la requête http renvoie un status Bad credential¹¹ ou Forbidden¹²

¹⁰ Interface Homme Machine

¹¹ Code d'erreur correspondant à de mauvais identifiants

¹² Code d'erreur correspondant à une tentative de connexion sans authentification

l'interface affiche une notification d'erreur, cela signifie que les identifiants sont incorrects ou inexistant.

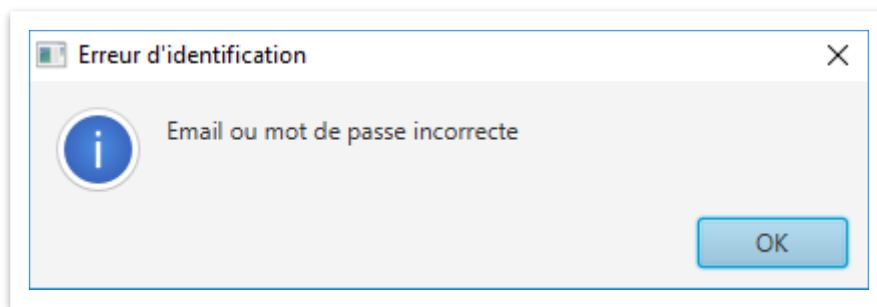


Figure 17 Notification d'erreur d'indentification

Menu Principal

Si la connexion est possible, le menu principal se charge en affichant les informations de l'entreprise qu'elle aura demandée à l'API et stockée dans une variable.

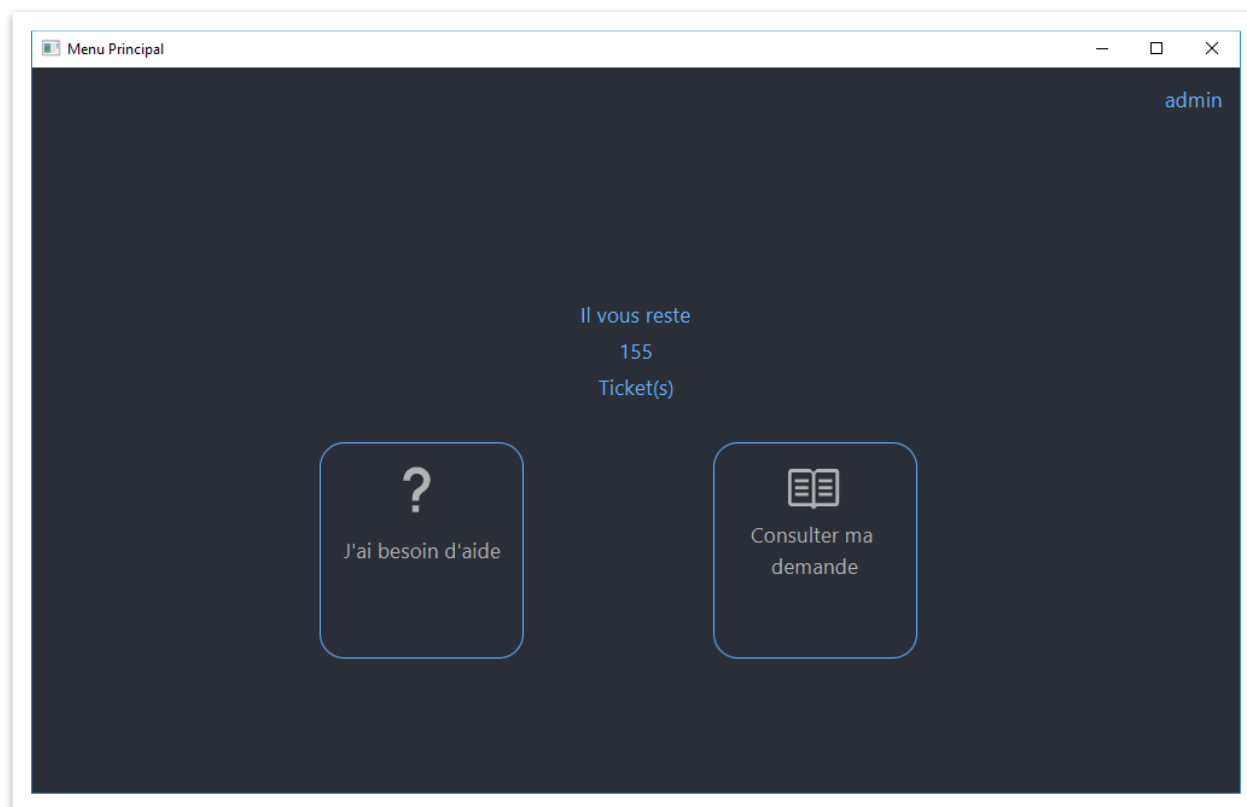


Figure 18 Menu principal de l'interface client

Le menu principal possède les éléments suivants :

- Le nom de l'entreprise,
- Le nombre de tickets restant à l'entreprise,
- Un bouton d'aide correspondant à la création d'une demande de dépannage,
- Un bouton pour afficher les informations de sa demande.

L'entreprise cliente est limitée à une seule demande à la fois, ainsi si ce dernier a déjà créé une demande, le bouton d'aide ne lui sera plus disponible, inversement si elle n'a pas encore créé de demande, elle ne pourra pas utiliser le bouton « Consulter ma demande ». L'interface possède une coloration du nombre de tickets si ce dernier est inférieur à six.

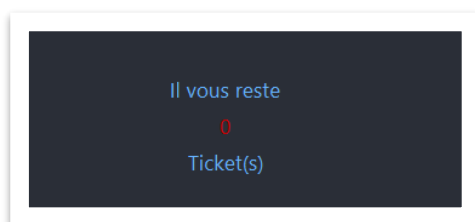


Figure 19 Coloration du nombre de tickets

Création d'une demande

Le bouton d'aide ouvre un formulaire de création de demande composé :

- Du titre de la demande résumant le problème,
- Du nom du demandeur dans l'entreprise cliente,
- D'un numéro de téléphone comme contact,
- D'une urgence entre « faible », « moyenne » et « haute »
- D'une description du problème,
- Un bouton pour ajouter des fichiers aux nombres de trois maximums ne dépassant pas la taille de 3 Mo¹³,
- Des boutons de confirmation et d'annulation.

¹³ 1 Mo correspond à 1 000 000 d'octets

Figure 20 Formulaire de création de demande

La totalité des champs sont obligatoires excepté l'ajout de fichiers. De plus les données saisies sont vérifiées, il est impossible de confirmer un numéro différent de 10 chiffres ou composé de lettres. Les fichiers peuvent être supprimés du formulaire en appuyant sur l'icône de suppression associée à leur répertoire machine.

Figure 21 Notification d'erreurs du formulaire

Dépannage informatique

En parallèle de la réalisation de l'application de gestion de tickets j'ai réalisé de nombreuses interventions de dépannage avec mon tuteur en tant qu'assistant. Les interventions se décomposaient en deux types :

- Les interventions à distance : réalisées par assistance téléphoniques, le but est de guider et conseiller le client afin de résoudre son problème. Les interventions de ce type peuvent être accompagnées d'une prise de contrôle à distance de l'ordinateur client avec le logiciel TeamViewer,
- Les interventions nécessitant un déplacement : quand le problème rencontré est d'ordre matériel ou qu'il nécessite des manipulations trop techniques, il est obligatoire de se rendre sur place afin de mettre en place des solutions.

L'ensemble des dépannages pouvaient porter sur des particuliers ou des entreprises.

Notions abordées

Au cours des différents dépannages de mon stage de nombreux concepts ont été abordés, ceux-ci pouvaient être d'ordre informatique ou pouvaient toucher d'autres domaines comme la communication. Cette partie a comme objectif de synthétiser les enseignements que j'ai reçus.

Solutions informatiques

Nombreuses sont les solutions que j'ai pu observer durant ces interventions, cette partie du stage a été un réel apprentissage. J'ai pu réaliser plusieurs types de solutions :

- Les solutions matérielles : elles portent généralement sur des problèmes de composants d'ordinateur ou sur des périphériques. Les solutions que j'ai pu étudier sont par exemple le montage complet d'un ordinateur ainsi que sa configuration logicielle, la modification de composants d'un ordinateur afin d'améliorer ses performances ou la configuration d'imprimantes et scanners,

- Les solutions relatives à la protection des données : ces solutions visent à sécuriser les données des entreprises et des particuliers par l'utilisation de plusieurs technologies. Cela porte sur l'installation d'antivirus performants, la copie et la sauvegarde des disques durs d'un ordinateur, le portage d'un ordinateur vers un système d'exploitation plus récent. J'ai également réalisé l'installation d'une solution de type NAS ¹⁴ dans un réseau d'entreprise, permettant la centralisation et la sécurisation des données de l'entreprise grâce à des sauvegardes régulières,
- Les solutions réseaux : ce sont toutes les solutions relatives aux problèmes de réseaux informatiques, ces cas d'interventions sont plus fréquents en entreprise que chez les particuliers. J'ai assisté à la vérification d'un switch, à la reconfiguration des ip statiques des machines d'un réseaux, à la mise en place de protocole SMB¹⁵ et IMAP¹⁶ ainsi qu'à l'intégration d'imprimantes réseaux. Pour finir j'ai abordé les concepts de « loadbalancing » et « d'agrégation de lien » permettant l'augmentation du débit montant et descendant d'un réseau.

Relation client

Les dépannages ont été aussi l'occasion d'en apprendre plus sur la communication avec la clientèle. Si cela peut paraître négligeable de prime abord, on observe que la relation avec le client est la partie la plus importante dans le dépannage. En effet, sur le marché du dépannage informatique il est très facile de trouver une autre prestation technique équivalente, cependant la différence entre deux prestations s'opère dans le lien créé avec le client.

Ainsi la communication avec le prestataire va déterminer la qualité du dépannage pour le demandeur. Ne possédant en général aucunes connaissances informatiques sur le motif de la prestation, le client se reposera en grande partie sur la relation entretenue avec le dépanneur pour juger de ses performances et exprimer son impression.

¹⁴ Serveur de stockage réseau

¹⁵ Protocole de partage de ressources

¹⁶ Protocole de courrier électronique

Un dépannage possède donc un aspect relationnel et psychologique important. Se contenter de réaliser l'aspect technique d'un dépannage n'est pas suffisant, il faut lui intégrer une phase d'écoute du client lui permettant d'expliquer son problème et d'évacuer la pression due aux problèmes informatiques. Une fois que le client s'est exprimé, il est préférable d'expliquer les démarches réalisées pendant le dépannage en imageant le plus possible pour faciliter la compréhension. Inonder un dépanné de termes techniques ne fera que susciter son incompréhension et une frustration.

Une certaine forme d'empathie fournie par le dépanneur permet ainsi d'établir une relation saine basée sur l'écoute et la communication permettant de réaliser un dépannage dans de bonnes conditions.

Business group

Le stage m'a également donné l'opportunité de participer à des réunions d'Azur Network, association présentée dans le début de ce rapport. Ces événements m'ont fait découvrir un nouvel aspect du monde de l'entreprise que je ne connaissais pas encore. Un ensemble d'entreprises se regroupent dans la but de collaborer dans des projets communs et d'entretenir leurs réseaux.

Dans une dynamique de groupe, les entreprises représentées par son dirigeant ou un associé, présentent tour à tour leurs actualités et leurs besoins. En entretenant de bonnes relations, Azur Network favorise ainsi les recommandations entre membres du réseau et fait collaborer de nombreux secteurs d'activités différents.

Difficultés rencontrées

La principale difficulté à laquelle j'ai dû me heurter est l'organisation et la gestion du temps lors de ma mission de développement. En effet, n'étant pas à temps plein dans le développement de ma mission, le planning prévisionnel que j'ai réalisé n'a pas été respecté, l'assistance en dépannage ayant occupé une grande partie de mon stage je n'ai pas pu consacrer autant de temps que prévu dans le développement.

J'ai cependant eu le temps de développer l'ensemble des fonctionnalités techniques du projet. La finalisation du projet consistera à l'élaboration de l'interface administrateur et à des modifications graphiques mineures côté interface cliente. De plus le temps passé en assistance de dépannage m'a permis d'acquérir beaucoup de nouvelles notions et se révèle être un des aspects les plus positif de mon stage.

L'apprentissage des concepts de sécurisation (SSL, authentification) ainsi que de la librairie JavaFx m'ont également causé quelques difficultés de compréhension ce qui a retardé mon emploi du temps. J'ai cependant pu maîtriser ces notions en y consacrant plus de temps et en me documentant davantage.

L'application principale n'étant pas finalisée je n'ai évidemment pas pu réaliser la mission facultative proposée par mon entreprise.

Conclusion

Durant les dix semaines de mon stage j'ai été amené à la réalisation d'un logiciel de gestion de tickets de dépannage pour des entreprises clientes d'AlloInformatique06. Les résultats de cette mission de développement sont très concluants, hormis la réalisation de l'interface administrateur, l'ensemble des fonctionnalités répondant aux besoins du cahier des charges sont implémentées. L'API est prête à être portée sur le serveur de l'entreprise, et l'application pour les clients n'a plus qu'à être finalisée graphiquement.

En addition de cette mission de développement j'ai également pu assister à de nombreux dépannages informatiques avec mon tuteur me permettant d'acquérir de nouvelles compétences complémentaires avec les enseignements du DUT informatique. Ces interventions m'ont aussi sensibilisé aux méthodes et aux différentes technologies de protections des données de l'entreprise tout en m'apportant des notions sur la communication et les relations clientes.

J'ai aussi pu assister à plusieurs réunions de business group m'introduisant de manière concrète au monde des entreprises et à leurs réseaux.

Du choix des technologies et de l'architecture au style graphique, ce stage de fin d'études m'a permis de réaliser une application complète en autonomie me confrontant à mes propres choix et erreurs. Cette mise en situation a été une expérience très enrichissante pour moi et me servira sans aucun doute pour mon avenir professionnel.

Pour finir, je poursuivrai dans un futur proche le développement de l'application afin de finaliser et d'apporter des améliorations à l'ensemble du projet que j'ai réalisé.

Tables des illustrations

Figure 1 Architecture globale de la mission.....	11
Figure 2 Schéma explicatif de la JVM	14
Figure 3 Architecture d'une application JavaFx	15
Figure 4 Un exemple de fichier FXML.....	16
Figure 5 Interface de recherche des icones FontAwesomeFX	17
Figure 6 Le scene builder permettant la réalisation d'interface JavaFX	18
Figure 7 Schéma de réalisation d'un cycle en V.....	21
Figure 8 Schéma entité/association initial.....	22
Figure 9 Configuration de Spring avec la base de données mysql.....	23
Figure 10 Représentation de la classe Entreprise qui sera convertie en table SQL	23
Figure 11 Représentation des échanges dans l'architecture	26
Figure 12 Format JSON d'une instance de la classe Entreprise	27
Figure 13 Déclaration du JpaRepository portant sur l'objet Entreprise.....	28
Figure 14 Gestion des champs obligatoires.....	32
Figure 15 Interface d'identification	32
Figure 16 Notification d'erreur d'indentification.....	33
Figure 17 Menu principal de l'interface client	33
Figure 18 Coloration du nombre de tickets.....	34
Figure 19 Formulaire de création de demande.....	35
Figure 20 Notification d'erreurs du formulaire	35